

ISPilot Project - Root Level Work Checkpoint

Status: ☒ API Production Ready + Root Services Operational

Date: 2026-08-29

API Endpoint: <https://ispilot-api-46y2f3tyja-uc.a.run.app>

Project Architecture Overview

ISPilot Multi-Agent Platform

intelligent-shelf-agents (ROOT)

AGENT LAYER (agents/)

- └ Coordinator Agent (ispilot_coordinator)
 - └ Delegates to specialized agents
- └ Shelf Analyst Agent (subagents/shelf_analyst)
 - └ Performance analysis, metrics, rankings
- └ Store Coach Agent (subagents/store_coach)
 - └ Recommendations, action plans

BUSINESS SERVICES LAYER (services/)

- └ AnalyticsService
 - └ Daily summary, metrics calculation
- └ StoreService
 - └ Store details, performance data
 - └ [Extensible for new domains]

DATA ACCESS LAYER (repositories/)

- └ BigQuery integration
 - └ NSG Gateway (Intelligent Shelf data)

DOMAIN MODELS (domain/)

- └ DailySummary, MetricSummary
- └ Store entities, Rankings
- └ Type-safe business objects

SHARED UTILITIES (common/)

- └ Model defaults (Gemini integration)
- └ Logging, config
- └ Cross-module utilities

└ VertexAgentClient imports
└ (ispilot-api/app/services/)



ispilot-api (DEPLOYMENT LAYER)

FastAPI REST API (/app)

- └ POST /chat - Main endpoint
- └ GET /health - Health check
- └ OpenAPI docs at /docs

Middleware Stack

- └ Audit Logging → Cloud Logging
- └ Authentication (OAuth + API Key)
- └ Request ID tracking
- └ CORS handling

Cloud Infrastructure

- └ Cloud Run deployment
- └ Cloud Logging (structured)
- └ Cloud Monitoring (6 custom metrics)
- └ Secret Manager (credentials)
- └ Firestore (session storage)

Current Status: Completed Work

Root Project (agents/, services/, repositories/, domain/)

Status: ☒ **Operational and Deployed**

Completed Capabilities:

1. ☒ Coordinator Agent - Multi-agent orchestration (ispilot_coordinator)
2. ☒ Shelf Analyst Agent - Performance analytics and metrics analysis
3. ☒ Store Coach Agent - Recommendations and action planning
4. ☒ AnalyticsService - Daily summary calculations
5. ☒ StoreService - Store detail retrieval
6. ☒ NSGGateway - BigQuery integration for Intelligent Shelf data
7. ☒ Domain Models - Type-safe business objects
8. ☒ Common Utilities - Shared logging, config, Gemini model defaults

Architecture:

- **Agent Engine:** Google Vertex AI Agent Engine (ID: 5375474415045705728)
- **Model:** Gemini 2.0 Flash (configured in common/models.py)
- **Data Source:** BigQuery (Intelligent Shelf metrics)
- **Supported Countries:** Chile (CL), Peru (PE)

Integration Points:

- Root agents are imported in `ispilot-api/app/services/vertex_agent_client.py`
 - VertexAgentClient orchestrates multi-agent conversations
 - Business services (analytics, store) can be called by API endpoints
-

API Project (ispilot-api/)

Status: ☒ **Production Ready - Sprint 2 Complete**

Sprint 2 Completion (31/31 checkpoints):

1. ☒ OpenAPI Documentation (Step 5) - 11/11 checkpoints
2. ☒ Enhanced Logging (Step 6) - 10/10 checkpoints
3. ☒ Cloud Monitoring Metrics (Step 7) - 10/10 checkpoints

Key Features:

- RESTful chat endpoint with session management
- Dual authentication (OAuth Bearer + API Key)
- 12 standardized error codes
- Comprehensive request/response logging to Cloud Logging
- 6 custom metrics for observability
- Graceful Firestore fallback (in-memory sessions)
- OpenAPI documentation with examples

Deployment:

- Cloud Run (us-central1)
 - Service Account: `sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com`
 - Workload Identity enabled
 - Health check at GET `/health`
-

Integration: Root → API Flow

User Request Journey

1. User sends chat message via API
POST `https://ispilot-api.../chat`

```
{  
  "user_id": "user123",  
  "message": "How is Talca Colin performing?",  
  "session_id": "session456"  
}
```
2. API Layer (ispilot-api)
 - └ Validates auth (OAuth or API Key)
 - └ Tracks request (audit logging, metrics)
 - └ Creates/loads session (Firestore or in-memory)
 - └ Invokes VertexAgentClient

3. Agent Orchestration (intelligent-shelf-agents/root)
 - └ Coordinator Agent receives message
 - └ Classifies intent (Analyst vs Coach)
 - └ Delegates to Shelf Analyst
 - └ Analyst calls AnalyticsService
 - └ AnalyticsService queries NSGGateway
 - └ NSGGateway → BigQuery
 - └ Results flow back through agent chain
4. Response Returns to API
 - └ API updates session
 - └ Logs response (metrics, Cloud Logging)
 - └ Returns ChatResponse with status
 - └ Client receives insight
5. Observability
 - └ Cloud Logging captures full lifecycle
 - └ Metrics recorded (latency, errors, cache hits)
 - └ Cloud Monitoring dashboard displays trends
 - └ Alerts trigger on SLO violations

Planned Sprints (Root + API Coordinated)

Sprint 3: Metrics Integration & Staging Deployment

Goal: Integrate observability into agents/services and deploy to staging

Root Tasks:

1. Add latency tracking to Coordinator Agent message handling
2. Add metrics recording to AnalyticsService queries
3. Add metrics recording to StoreService queries
4. Add error tracking to subagents (shelf_analyst, store_coach)
5. Document agent performance baselines
6. Add agent-level logging to Cloud Logging
7. Test agent chain with metrics enabled

API Tasks:

1. Integrate `track_vertex_latency()` into VertexAgentClient
2. Integrate `track_cache_operation()` into SessionService
3. Deploy to staging environment
4. Verify Cloud Logging entries
5. Set up Cloud Monitoring dashboard
6. Configure alert policies
7. Test SLO compliance with real requests

Coordination:

- Root: Ensure agents emit metrics within latency budget (< 30s per Vertex call)
- API: Measure end-to-end latency and cache performance
- Dashboard: Visualize agent latency vs API latency breakdown

Estimated Checkpoints: 15-18

Sprint 4: Agent Expansion & Production Hardening

Goal: Add new agents and prepare for production load

Root Tasks:

1. Implement Executive Agent (executive-level summaries)
2. Implement Root Cause Agent (failure analysis)
3. Add sub-agent error handling and fallbacks
4. Add agent timeout policies (prevent hanging)
5. Load testing (concurrent agent requests)
6. Chaos testing (BigQuery timeouts, NSG unavailable)
7. Agent performance tuning

API Tasks:

1. Security review (OAuth tokens, secret rotation)
2. Load testing (concurrent chat requests)
3. Chaos testing (agent timeouts, Firestore unavailable)
4. Rate limiting implementation
5. Connection pooling optimization
6. Production deployment procedures
7. Rollback procedures

Coordination:

- Define agent response time budgets (SLA)
- Set up concurrent agent orchestration
- Test multi-agent conversations with API latency

Estimated Checkpoints: 12-15

Sprint 5: Advanced Observability & Tracing

Goal: Deep visibility into agent decision chains and data flow

Root Tasks:

1. Structured agent logging (decisions, routing, tool calls)
2. Agent execution tracing (which subagent was chosen, why)
3. Data flow logging (BigQuery query details)
4. Agent performance profiling

5. Cost tracking per agent and query
6. Custom agent dashboards in Cloud Monitoring

API Tasks:

1. Distributed tracing (Cloud Trace integration)
2. Trace propagation through agent chain
3. Request correlation (trace ID ↔ session ID ↔ request ID)
4. Custom dashboards (end-to-end request flow)
5. SLO reporting automation
6. Weekly metrics summaries

Coordination:

- Unified trace view: API request → Agent decisions → BigQuery queries
- Understand latency bottlenecks at each layer
- Identify optimization opportunities

Estimated Checkpoints: 10-12

Sprint 6: Enterprise Integrations

Goal: Enable consumption through multiple channels beyond API

Root Tasks:

1. Teams Bot integration (agent responses in Teams)
2. Copilot Studio connector (agent as Copilot plugin)
3. Scheduled reports (email daily summaries)
4. Webhook integration (push notifications)
5. Batch analysis mode (analyze multiple stores)
6. Agent fine-tuning based on user feedback

API Tasks:

1. Webhook endpoints (delivery status, events)
2. Batch processing endpoint
3. Export formats (CSV, PDF, PowerPoint)
4. Scheduled task integration (Cloud Tasks)
5. User preferences storage (Firestore)
6. Multi-user conversation threads

Coordination:

- Ensure agent outputs work across all formats
- Consistent styling/formatting for different channels
- Unified session/context management

Estimated Checkpoints: 12-16

Technical Dependencies

Root Project Dependencies

- **google-cloud-bigquery** - NSSGateway queries
- **google-adk** - Agent framework
- **google-cloud-logging** - Structured logging
- **pydantic** - Domain models

API Project Dependencies

- **fastapi==0.115.0** - REST framework
- **pydantic==2.9.2** - Request validation
- **google-cloud-logging** - Audit logging
- **google-cloud-monitoring** - Custom metrics
- **google-cloud-firestore** - Session storage
- **google-cloud-secret-manager** - Credentials

Shared Infrastructure

- **GCP Project:** corp-stro-salesinventory-prod
- **Service Account:** sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com
- **Vertex AI Engine ID:** 5375474415045705728 (us-central1)
- **BigQuery:** Data source for Intelligent Shelf metrics

Resume Instructions: Continuing Work

When Returning to This Project

1. Check Project Status:

- Review this WORK_CHECKPOINT.md
- Check ispiilot-api/WORK_CHECKPOINT.md for API-specific status
- Verify git log: `git log --oneline -5`

2. Set Up Environment (Root):

```
cd intelligent-shelf-agents
python -m venv venv
source venv/bin/activate # or venv\Scripts\activate on Windows
pip install -r requirements.txt
pip install -r requirements.lock.txt
```

3. Set Up Environment (API):

```
cd ispiilot-api
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Verify Current State:

- Root agents operational: `python -c "from agents.coordinator.agent import root_agent; print(root_agent.name)"`
- API running: `uvicorn app.main:app --reload --port 8080`
- Check Cloud Run deployment: `gcloud run services list --project corp-stro-salesinventory-prod`

5. Understanding Code Structure:

- **agents/coordinator/** - Main orchestration logic
- **services/** - Business logic (analytics, store info)
- **repositories/** - Data access (BigQuery)
- **domain/** - Type definitions
- **common/** - Shared utilities
- **ispilot-api/app/** - FastAPI endpoints
- **ispilot-api/docs/** - Technical documentation

6. For Sprint 3 Start:

- Review metrics definitions: `ispilot-api/app/utils/metrics_definitions.py`
- Understand metrics decorators: `ispilot-api/app/utils/observability.py`
- Plan agent-level metrics (latency, error rates by agent)
- Create staging deployment configuration

Key Metrics & SLOs

API Layer SLOs

- **Request Latency:** p99 < 2000ms
- **Error Rate:** < 5%
- **Cache Hit Rate:** > 70%
- **Vertex API Latency:** < 30 seconds

Agent Layer SLOs (To Be Defined in Sprint 3)

- **Coordinator Routing Time:** < 2s
 - **Agent Execution Time:** < 25s (budget for Vertex timeout)
 - **Subagent Delegation Success:** > 95%
 - **Tool Call Success Rate:** > 98%
-

File Structure Reference

```
intelligent-shelf-agents/
├── WORK_CHECKPOINT.md      ← You are here
├── README.md               ← Project vision & architecture
├── pyproject.toml          ← Root Python config
├── requirements.lock.txt    ← Locked dependencies
├──
├── agents/                 ← Agent orchestration
│   ├── coordinator/
│   │   ├── agent.py        ← Coordinator agent definition
│   │   ├── subagents/
│   │   │   ├── shelf_analyst/
│   │   │   │   └── store_coach/
│   │   │   └── requirements.txt
│   ├──
├── services/               ← Business logic
│   ├── analytics_service.py ← Analytics calculations
│   ├── store_service.py     ← Store info
│   └── __init__.py
├── repositories/           ← Data access
│   └── [BigQuery, NSG Gateway]
├── domain/                 ← Type definitions
│   ├── summary.py          ← DailySummary, MetricSummary
│   └── [Business entities]
├── common/                 ← Shared utilities
│   ├── models.py           ← get_default_model()
│   └── [Config, logging]
├── gateways/               ← External integrations
│   └── nsg_gateway.py       ← NSG/BigQuery
├── ispirot-api/            ← REST API deployment
│   ├── WORK_CHECKPOINT.md  ← API-specific checkpoint
│   ├── README.md
│   ├── DEPLOYMENT.md
│   ├── app/
│   │   ├── main.py         ← FastAPI setup
│   │   ├── api/
│   │   │   ├── chat.py     ← Chat endpoint
│   │   ├── services/
│   │   │   └── vertex_agent_client.py ← Orchestration
│   │   ├── middleware/
│   │   │   └── audit.py     ← Logging & auth
│   │   ├── models/
│   │   │   ├── responses.py ← Request/response models
│   │   │   └── errors.py    ← Error definitions
│   │   └── utils/
```

```
├── metrics.py          ← Cloud Monitoring client
├── metrics_definitions.py ← Metric schemas
├── observability.py ← Decorators & context managers
├── docs/
│   ├── AUTHENTICATION_CHANGES.md
│   ├── PERMISSIONS_AND_IAM.md
│   ├── SESSION_SERVICE_FIRESTORE_FALLBACK.md
│   └── DEPLOY_SCRIPT_CHANGES.md
└── tests/
```

Summary

ISPilot Project Status: ☒ **FULLY OPERATIONAL**

- ☒ Root multi-agent platform deployed and running
- ☒ API production-ready on Cloud Run
- ☒ Comprehensive logging and monitoring infrastructure
- ☒ 31/31 API validation checkpoints passing
- ☒ Zero regressions in all components
- ☒ Ready for Sprint 3 (Metrics Integration & Staging)

Next Priority: Sprint 3 - Integrate observability decorators into agents and API, then deploy to staging for validation.

Contact Point: WORK_CHECKPOINT.md (root level for project overview) + ispirot-api/WORK_CHECKPOINT.md (API-specific details)